Python

Drugi wykład

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Przepraszam was za to,
że będę do was mówić przez „wy" :-)

Niektóre objaśnienia i przykłady pochodzą
z kursu dr. hab. Grzegorza Góralskiego
Jego kurs jest dostępny pod adresem
https://ggoralski.gitlab.io/python-wprowadzenie/

Proszę robić notatki :-)

Plan na dziś

Dzisiaj przedstawię wam:

+ Funkcję **print**
+ Funkcje
+ Łańcuchy znaków, czyli teksty
+ Instrukcję przypisania
+ Słowa kluczowe
+ Funkcję **input**
+ Indeksowanie łańcuchów
+ Wycinki łańcuchów
+ Dodawanie łańcuchów
+ Metody łańcuchów
+ Instrukcję warunkową **if**
+ Operatory porównania
+ Operatory **in** i **not in**
+ Operatory logiczne
+ I pliki tekstowe

Zadanie na rozgrzewkę:
Proszę narysować plik
Każda odpowiedź jest dobra :-)

Wrócimy do plików później

Funkcja print

Umówmy się

Umówmy się, że na slajdach prezentacji:

+ fragmenty programów w Pythonie będą zielone
+ wyniki, które te programy wypisują na ekranie będą pomarańczowe
+ dane, które wprowadza użytkownik programu, będą jasnoniebieskie

Co to jest funkcja w językach programowania?

Funkcja to osobny fragment programu, który działa na określonych danych

Każdą funkcję możemy wywołać, czyli spowodować jej wykonanie. Za każdym razem, gdy wywołujemy funkcję, możemy przekazać jej inne dane

Funkcja print

Funkcja print wypisuje tekst

Tak wywołujemy (my i Python) funkcję print:

```
print('Witaj, piękny świecie :-)')
Witaj, piękny świecie :-)
```

Funkcja print

```
print('Witaj, piękny świecie :-)')
```
```
Witaj, piękny świecie :-)
```

Aby wywołać funkcję **print**, napisaliśmy:
+ nazwę funkcji
+ nawiasy ( ), a wewnątrz nich argument funkcji

Wywołaliśmy funkcję **print** z jednym argumentem. Tym argumentem jest tekst Witaj, piękny świecie :-)

Funkcje

Z czego składa się funkcja?

+ Każda funkcja ma nazwę

+ Każda funkcja może mieć dane wejściowe. Mówimy na te dane wejściowe argumenty funkcji

+ Każda funkcja ma kod, czyli instrukcje, które są wykonywane przez program, kiedy wywołujemy tę funkcję

Wywoływanie funkcji

Aby program wykonał funkcję, powinniśmy
(my i program) wywołać tę funkcję

```
print('2 + 2 =', 2+2)
```

```
2 + 2 = 4
```

Wywoływanie funkcji

```
print('2 + 2 =', 2+2)
2 + 2 = 4
```

Aby wywołać funkcję, piszemy kolejno:
+ nazwę funkcji
+ nawiasy ( )
+ a wewnątrz tych nawiasów argumenty funkcji, rozdzielone przecinkami ,

Definiowanie funkcji

Każda funkcja jest gdzieś zdefiniowana, czyli gdzieś istnieje jej definicja

```
def wypisz_sumę(a, b):
    print(a, '+', b, '=', a+b)
```

Definiowanie funkcji

```
def wypisz_sumę(a, b):
    print(a, '+', b, '=', a+b)
```

Definicja funkcji określa:

+ nazwę funkcji
+ argumenty funkcji
+ kod funkcji

Definiowanie funkcji

Kiedy Python widzi definicję funkcji, nie wykonuje tej funkcji, tylko zapamiętuje jej nazwę, argumenty i kod

Dopiero potem możemy (my i program) wywołać tę funkcję, czyli podać jej nazwę i argumenty, aby wykonać jej kod

Definiowanie funkcji

Kiedy wywołujemy funkcję, powinniśmy podać tyle samo argumentów, co wtedy, gdy ją definiowaliśmy

Kiedy program wywołuje funkcję, przypisuje wartości jej argumentów w miejscu wywołania do argumentów podanych w jej definicji

Definiowanie i wywoływanie funkcji — przykład

```
def wypisz_sumę(a, b):
    print(a, '+', b, '=', a+b)

wypisz_sumę(2, 2)
wypisz_sumę(3, 6)
```

```
2 + 2 = 4
3 + 6 = 9
```

Funkcje wbudowane, funkcje biblioteczne i funkcje definiowane przez użytkownika

Funkcje można podzielić na trzy grupy:

1. Definicje najpotrzebniejszych funkcji są wbudowane w Pythona. Taka jest na przykład funkcja **print**

2. Definicje wielu innych funkcji są umieszczone w bibliotekach funkcji.
Slogan Pythona „baterie są w zestawie" oznacza, że biblioteki funkcji dostarczane wraz z Pythonem przydają się w wielu zastosowaniach

Funkcje wbudowane, funkcje biblioteczne i funkcje definiowane przez użytkownika

3. Definicje pozostałych funkcji są tworzone przez programistów

Funkcje zwracające wynik i funkcje z efektami ubocznymi

Są dwa rodzaje funkcji:

+ Takie funkcje, które zwracają jakiś wynik. Takie są na przykład funkcje matematyczne z biblioteki math: math.sin, math.cos i inne

+ Takie funkcje, które nie zwracają wyniku, ale za to zmieniają coś dookoła siebie, na przykład na ekranie. Taka jest na przykład funkcja **print**

Łańcuchy znaków, czyli teksty

Łańcuchy znaków, czyli teksty

Aby komputer mógł odróżnić teksty, które wypisuje na ekranie, od innych części programu, umieszczamy każdy tekst wewnątrz cudzysłowów:
+ pojedynczych 'to jest przykład tekstu'
+ albo podwójnych "to też jest przykład tekstu"

Cudzysłowy wokół tekstu nie wchodzą w skład tekstu, tylko oznaczają jego początek i koniec

Łańcuchy znaków, czyli teksty

Inna nazwa tekstu to łańcuch znaków, w skrócie łańcuch

(Po angielsku łańcuch znaków nazywa się „string", czyli „sznurek")

Znakiem może być litera, cyfra, znak przestankowy

Są nawet białe znaki: spacja, znak tabulacji, znak nowej linii. Powiem o nich potem

Łańcuchy znaków, czyli teksty

Cudzysłowy ' lub " oznaczają początek i koniec łańcucha. Oba rodzaje cudzysłowów są dobre

Wewnątrz łańcucha możemy umieszczać cudzysłowy, ale cudzysłowy wewnątrz łańcucha muszą być inne niż te cudzysłowy, które oznaczają początek i koniec tego łańcucha

Łańcuchy znaków, czyli teksty

Wewnątrz łańcucha możemy umieszczać cudzysłowy, ale cudzysłowy wewnątrz łańcucha muszą być inne niż te cudzysłowy, które oznaczają początek i koniec tego łańcucha

```
print('Autorem książki "Trzej muszkieterowie" jest Alexandre Dumas.')
```
Autorem książki "Trzej muszkieterowie" jest Alexandre Dumas.

```
print("D'Artagnan był muszkieterem.")
```
D'Artagnan był muszkieterem.

Łańcuchy znaków, czyli teksty

A jednak możemy umieścić w łańcuchu taki sam cudzysłów jak te cudzysłowy, które otaczają ten łańcuch, pod warunkiem, że poprzedzimy ten cudzysłów znakiem lewego ukośnika \

```
print("\"Monty Python i Święty Graal\" to komedia filmowa z 1975 roku.")
```

"Monty Python i Święty Graal" to komedia filmowa z 1975 roku.

(Po angielsku lewy ukośnik nazywa się „backslash")

Łańcuchy znaków, czyli teksty

Dzięki lewemu ukośnikowi \ możemy wstawiać do łańcuchów takie znaki, które trudno zapisać bezpośrednio, na przykład:

+ \' w zapisie łańcucha oznacza pojedynczy cudzysłów ' w łańcuchu
+ \" w zapisie łańcucha oznacza podwójny cudzysłów " w łańcuchu
+ \\ w zapisie łańcucha oznacza lewy ukośnik \ w łańcuchu
+ \n w zapisie łańcucha oznacza znak nowej linii w łańcuchu
+ \t w zapisie łańcucha oznacza znak tabulacji w łańcuchu

Łańcuchy znaków, czyli teksty

+ \' w zapisie łańcucha oznacza pojedynczy cudzysłów ' w łańcuchu
+ \" w zapisie łańcucha oznacza podwójny cudzysłów " w łańcuchu
+ \\ w zapisie łańcucha oznacza lewy ukośnik \ w łańcuchu
+ \n w zapisie łańcucha oznacza znak nowej linii w łańcuchu
+ \t w zapisie łańcucha oznacza znak tabulacji w łańcuchu

```
print('raz\ndwa\ntrzy')
raz
dwa
trzy
```

Instrukcja przypisania

Instrukcja przypisania

```
tekst = 'Ahoj, krásný světe :-)'
print(tekst)
```
```
Ahoj, krásný světe :-)
```

Instrukcja przypisania nazwa = obiekt wiąże nazwę z obiektem. Kiedy potem używamy tej nazwy, jest tak, jakbyśmy używali tego obiektu. Mówimy wtedy, że używamy zmiennej

Instrukcja przypisania

Program może zmieniać wartość zmiennej
W każdej chwili zmienna ma tylko jedną wartość

```
tekst = 'Ahoj, krásný světe :-)'
print(tekst)
tekst = 'Witaj, piękny świecie :-)'
print(tekst)
```

```
Ahoj, krásný světe :-)
Witaj, piękny świecie :-)
```

Nazwy zmiennych i funkcji

Nazwy

Nazwa zmiennej lub funkcji może się składać z:
+ liter, w tym polskich liter
+ cyfr od 0 do 9
+ znaków podkreślenia _

Nazwa zmiennej lub funkcji nie może zawierać spacji

Nazwa zmiennej lub funkcji nie może się zaczynać od cyfry, czyli może się zaczynać od litery albo od znaku podkreślenia

Nazwy

W nazwach odróżniamy duże litery od małych liter

Na przykład **nazwisko**, **Nazwisko** i **NAZWISKO** to trzy różne nazwy

Dobre zwyczaje

Dobrzy programiści mają dobre zwyczaje:

+ zaczynamy nazwę każdej zmiennej i funkcji małą literą
+ łączymy nazwy złożone z wielu wyrazów znakami podkreślenia _
+ nazywamy zmienne rzeczownikami, na przykład **gatunek**, **liczba_prób**
+ nazywamy funkcje czasownikami w trybie rozkazującym, na przykład **wczytaj_gatunek**, **oblicz_podatek**

Słowa kluczowe

Słowa kluczowe

Znaczenie każdej nazwy w programie ustalają programiści

Słowa kluczowe wyglądają podobnie do nazw, ale znaczenie słów kluczowych ustalili twórcy Pythona. Nie da się zmienić tego znaczenia

W Pythonie jest trzydzieści kilka słów kluczowych, na przykład **if**, **for**, **def**

W Pythonie nie możemy używać słów kluczowych jako nazw

Umówmy się, że na slajdach słowa kluczowe będą **podkreślone**

Komentarze

Każdy komentarz to część programu przeznaczona dla czytelników programu. Python nie czyta komentarzy

Umówmy się, że komentarze na slajdach będą ciemnoniebieskie

Każdy komentarz zapisujemy od znaku kratki do końca linii

Łańcuchy, nazwy, słowa kluczowe
Podsumowanie

Łańcuchy, nazwy, słowa kluczowe — podsumowanie

```
'To jest łańcuch znaków, czyli tekst'

"To też jest łańcuch znaków"

'To są znaki specjalne:\nznak nowej linii, \\ lewy ukośnik'

to_jest_nazwa

def if for  # To są słowa kluczowe
```

Funkcja print — uzupełnienie

Funkcja print — uzupełnienie

Funkcja print może mieć 0 argumentów, 1 argument albo więcej niż 1 argument. Kiedy wypisuje te argumenty, oddziela je spacjami. Kiedy wypisze wszystkie argumenty, kończy pisać w tej linii. Programiści mówią: „przechodzi do nowej linii"
Kiedy następny raz wywołamy funkcję print, ta funkcja zacznie pisać od następnej linii

Funkcja print — uzupełnienie

```
print('Cześć :-)')
print()  # Wypisz pustą linię
print('Jak się masz?')
```

```
Cześć :-)

Jak się masz?
```

Funkcja print — uzupełnienie

```
# Wypisz dwa argumenty, oddzielając je od siebie
# pojedynczymi znakami spacji,
# a potem przejdź do nowej linii
miasto = 'Kraków'
print(miasto, 'to ładne miasto')
Kraków to ładne miasto
```

Funkcja input

Funkcja input

Dzięki funkcji input program wczytuje teksty
z klawiatury

Funkcja input:
+ wyświetla swój argument na ekranie
+ czeka, aż użytkownik wpisze tekst i naciśnie klawisz
Enter
+ zwraca wpisany tekst

Funkcja input

Dzięki funkcji **input** program wczytuje teksty
z klawiatury

```
imię = input('Jak masz na imię? ')
print(imię, 'to ładne imię :-)')
Jak masz na imię? Eleonora
Eleonora to ładne imię :-)
```

Postęp wykładu

Przedstawiłem wam:
+ Funkcję **print**
+ Funkcje
+ Łańcuchy znaków, czyli teksty
+ Instrukcję przypisania
+ Słowa kluczowe
+ I funkcję **input**

Teraz omówię:
+ Indeksowanie łańcuchów
+ Wycinki łańcuchów
+ Dodawanie łańcuchów
+ I metody łańcuchów

Proszę mi zadać pytanie :-)

Jaki polski wyraz brzmi najładniej w Twoich uszach?

Według pewnego Anglika najładniejszy polski wyraz to nie miłość

Według tego Anglika najładniej brzmi wyraz

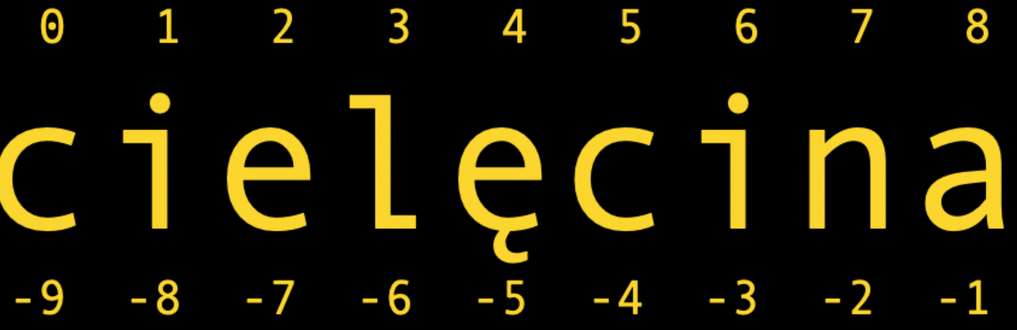
cielęcina :-)

Indeksowanie łańcuchów

Indeksowanie łańcuchów

Każdy znak łańcucha ma swój numer, czyli indeks

Odczytujemy pojedynczy znak łańcucha, podając:
+ nazwę tego łańcucha albo po prostu ten łańcuch
+ indeks tego znaku w nawiasach kwadratowych [ ]



Indeksowanie łańcuchów

Pierwszy znak ma indeks 0
Drugi znak ma indeks 1
Trzeci znak ma indeks 2
i tak dalej…



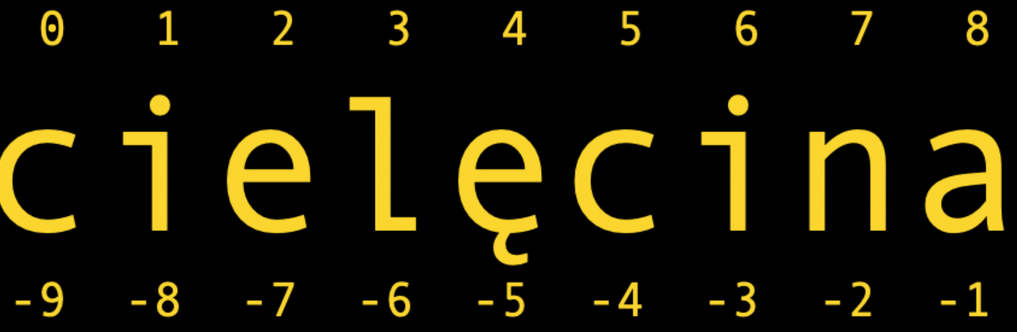

```
tekst = 'cielęcina'
print(tekst[2])
```
```
e
```

Programiści tak czytają zapis
łańcuch[2]:
„łańcuch pod indeksem dwa"

Indeksowanie łańcuchów

Dla wygody możemy liczyć
indeksy od końca łańcucha,
używając liczb ujemnych

Ostatni znak ma indeks -1
Przedostatni znak ma indeks -2
i tak dalej…




```
tekst = 'cielęcina'
print(tekst[-2], tekst[-5])
```

```
n ę
```

Indeksowanie łańcuchów

```
tekst = 'cielęcina'
```

Próba użycia indeksu
większego lub równego
długości łańcucha
powoduje wyjątek

```
print(tekst[100])
```

```
IndexError
```



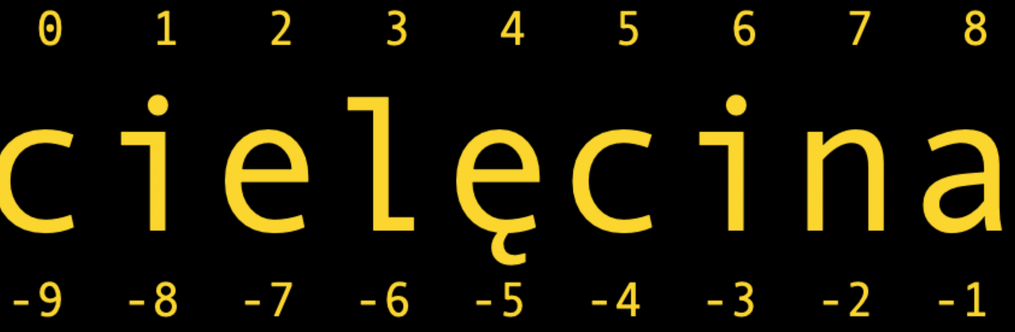
Indeksowanie łańcuchów

```
tekst = 'cielęcina'
```

Próba użycia indeksu mniejszego niż minus długość łańcucha też powoduje wyjątek

```
print(tekst[-17])
```

```
IndexError
```



Jak wygląda wyjątek?

```
print(tekst[-17])
```

Jeśli coś spowodowało wyjątek, to Python wypisze na ekranie raport o tym wyjątku

Na dole raportu znajdziemy opis tego wyjątku

```
IndexError: string index out of range
```

Wycinki łańcuchów

Wycinki łańcuchów

Odczytujemy dowolnie wiele sąsiednich znaków łańcucha, czyli wycinek łańcucha, podając w nawiasach kwadratowych [ ] indeksy początku wycinka i końca wycinka, rozdzielone dwukropkiem :




```
tekst = 'cielęcina'
print(tekst[0:5])
cielę
```

Wycinki łańcuchów

Najwygodniej zapamiętać, że
indeks początku wycinka
i indeks końca wycinka to
indeksy miejsc między
znakami łańcucha

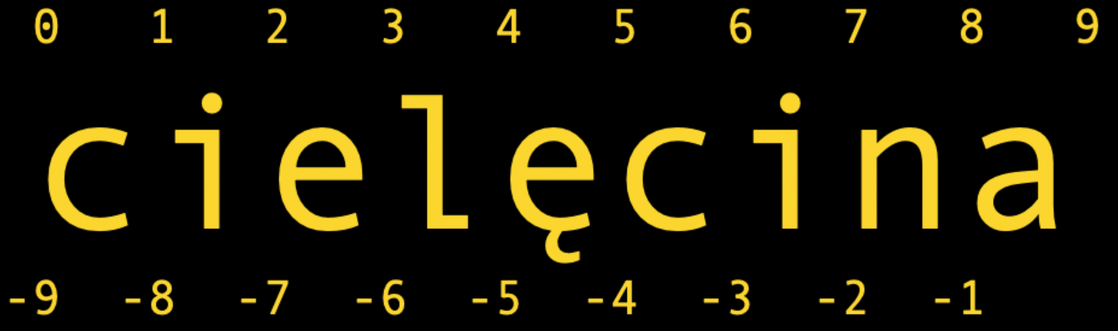



```
tekst = 'cielęcina'
print(tekst[5:7], tekst[-4:-2])
```

```
ci ci
```

Wycinki łańcuchów

Gdy w wycinku podamy taki indeks początku albo taki indeks końca, któremu odpowiada znak poza łańcuchem, to wyjątek nie wyskoczy




```
tekst = 'cielęcina'
print(tekst[-300:5], tekst[-300:300])
```

```
cielę cielęcina
```

Wycinki łańcuchów

Nie ma też wyjątku, gdy w wycinku podamy indeks początku większy od indeksu końca

```
tekst = 'cielęcina'
print(tekst[5:2])
```

Wtedy wycinek jest pusty



Wycinki łańcuchów

W wycinkach możemy pomijać indeks początku. Wtedy Python przyjmuje, że indeks początku to zero




```
tekst = 'cielęcina'
print(tekst[:5], tekst[:-4])
```

```
cielę cielę
```

Wycinki łańcuchów

W wycinkach możemy pomijać indeks końca. Wtedy Python przyjmuje, że indeks końca to długość łańcucha




```
tekst = 'cielęcina'
print(tekst[7:], tekst[-2:])
print(tekst[:])
```

```
na na
cielęcina
```

Dodawanie łańcuchów

Dodawanie łańcuchów

Łańcuchy możemy dodawać do łańcuchów znakiem plus +:

```
rebus = 'cielęcina'
print('m' + rebus[1:5] + ' szp' + rebus[-3:] + 'k')
```

Dodawanie łańcuchów

Łańcuchy możemy dodawać do łańcuchów znakiem plus +:

```
rebus = 'cielęcina'
print('m' + rebus[1:5] + ' szp' + rebus[-3:] + 'k')
```

```
mielę szpinak
```

Metody łańcuchów

Metody

Metoda to taka funkcja, która jest powiązana z obiektem

Metody obiektu wywołuje się, pisząc kolejno:
+ nazwę tego obiektu albo po prostu ten obiekt
+ kropkę .
+ nazwę metody
+ nawiasy ( ), a wewnątrz nich argumenty metody, rozdzielone przecinkami ,

Umówmy się, że na slajdach nazwy metod będą miały na początku kropkę

Łańcuchy — metoda .upper

Każda metoda łańcuchów zwraca jakiś wynik

Na przykład metoda .upper zwraca nowy łańcuch. W tym łańcuchu wszystkie małe litery są zamienione na wielkie litery

Łańcuchy — metoda .upper

Metoda .upper zwraca nowy łańcuch. W tym łańcuchu wszystkie małe litery są zamienione na wielkie litery

```
tekst = 'Cielęcina 123'
nowy_tekst = tekst.upper()
print(nowy_tekst)
CIELĘCINA 123
```

Łańcuchy — metoda .lower

Metoda .lower zwraca nowy łańcuch. W tym łańcuchu wszystkie wielkie litery są zamienione na małe litery

```
tekst = 'Dziennik Polski'
tekst = tekst.lower()
print(tekst)
```

dziennik polski

Łańcuchy — metoda .strip

Metoda .strip zwraca nowy łańcuch bez początkowych
i końcowych białych znaków

Białe znaki to:

+ spacje
+ znaki tabulacji \t
+ znaki nowej linii \n

Łańcuchy — metoda .strip

Metoda .strip zwraca nowy łańcuch bez początkowych i końcowych białych znaków

```
tekst = '\tAnna Kowalska   \n'
print('[' + tekst.strip() + ']')
```

```
[Anna Kowalska]
```

Typ logiczny **bool**

Obiekty, które dotychczas widzieliśmy, to łańcuchy znaków

Oprócz nich są obiekty innych typów, między innymi typu logicznego, czyli typu **bool**

Są 2 obiekty typu logicznego: **True**, czyli prawda, i **False**, czyli nieprawda

(Typ logiczny nazywa się po angielsku „Boolean type”, czyli typ Boolowski, od nazwiska angielskiego matematyka George'a Boole'a)

Łańcuchy — metoda .startswith

Metoda .startswith sprawdza, czy dany łańcuch znaków zaczyna się od innego łańcucha znaków. Zwraca True albo False

```
tekst = 'cielęcina'
print(tekst.startswith('ciel'))
print(tekst.startswith('ć'))
```

```
True
False
```

Łańcuchy — metoda .endswith

Metoda .endswith sprawdza, czy dany łańcuch znaków kończy się innym łańcuchem znaków. Zwraca True albo False

```
tekst = 'cielęcina'
print(tekst.endswith('ć'))
print(tekst.endswith('na'))
False
True
```

Przypisanie wielu wartości do wielu zmiennych

Możemy naraz przypisywać wiele wartości do wielu zmiennych pod warunkiem, że wartości jest tyle samo, co zmiennych

```
imię, drugie_imię, nazwisko = 'Anna', 'Maria', 'Kowalska'
```

Próba takiego przypisania, w którym wartości nie jest tyle samo, co zmiennych, powoduje wyjątek

Łańcuchy — metoda .split

Metoda .split dzieli łańcuch na części złożone z takich znaków, które nie są białymi znakami

```
dane_osobowe = ' Anna Maria Kowalska\n'
imię, drugie_imię, nazwisko = dane_osobowe.split()
```

Łańcuchy — metoda .split

```
dane_osobowe = ' Anna Maria Kowalska\n'
imię, drugie_imię, nazwisko = dane_osobowe.split()
print(imię)
print(drugie_imię)
print(nazwisko)
```

```
Anna
Maria
Kowalska
```

Funkcja wbudowana len

Funkcja **len** (skrót od "length") zwraca długość łańcucha

```
tekst = 'pies'
print(len(tekst))
tekst = ''  # To są dwa pojedyncze cudzysłowy
print(len(tekst))
```

```
4
0
```

Indeksowanie, wycinki, dodawanie, metody łańcuchów Podsumowanie

Indeksowanie, wycinki, dodawanie, metody łańcuchów — podsumowanie

```
tekst = 'łańcuch'
print(tekst[0])
print(tekst[1:3])
print(tekst[:4])
print(tekst[-3:])
print(tekst[-3:] + 'o')
```

```
print('Python'.upper())
print('Python'.lower())
print('Python\n'.strip())
print('Python'.startswith('Py'))
print('Python'.endswith('hon'))
imię, nazwisko = 'Jan Kot\n'.split()
print(len('Python'))
```

Postęp wykładu

Przedstawiłem wam:
+ Funkcję **print**
+ Funkcje
+ Łańcuchy znaków, czyli teksty
+ Instrukcję przypisania
+ Słowa kluczowe
+ Funkcję **input**
+ Indeksowanie łańcuchów
+ Wycinki łańcuchów
+ Dodawanie łańcuchów
+ I metody łańcuchów

Teraz omówię:
+ Instrukcję warunkową **if**
+ Operatory porównania
+ Operatory **in** i **not in**
+ I operatory logiczne

Proszę mi zadać pytanie :-)

Instrukcja warunkowa if

Instrukcja warunkowa if

Instrukcja warunkowa to:
+ Słowo kluczowe if
+ Wyrażenie logiczne. Jego wynikiem powinno być True albo False
+ Dwukropek :
+ Nowa linia
+ Kilka spacji albo znak tabulacji, tak zwane wcięcie

Wcięcia są ważne :-)

+ Instrukcje przesunięte w prawo. Te instrukcje wykonają się tylko wtedy, gdy wartością wyrażenia logicznego jest True

Instrukcja warunkowa if

```
test = True
if test:
    print('To prawda:')
    print('zmienna test ma wartość True')
```

```
To prawda:
zmienna test ma wartość True
```

Instrukcja warunkowa if

```
test = True
if test:
    print('Zmienna test ma wartość True')
Zmienna test ma wartość True

test = False
if test:
    # Instrukcja poniżej się nie wykona :-)
    print('Zmienna test ma wartość True')
```

Operatory

Operatory

Operatory to takie symbole lub słowa kluczowe, dzięki którym program wykonuje działania na obiektach

Najpierw poznacie operatory dwuargumentowe. Każdy operator dwuargumentowy ma dwa argumenty. Lewy argument operatora znajduje się po lewej stronie operatora. Prawy argument operatora znajduje się po prawej stronie operatora :-)

Operatory porównania

Operatory porównania

Dzięki operatorowi == program porównuje argumenty tego operatora. Wynik operatora == to:
+ **True**, gdy oba jego argumenty są równe
+ **False**, gdy oba jego argumenty nie są równe

Operator == czytamy „równa się”

Operatory porównania

```
tekst1 = input('Wpisz cokolwiek: ')
tekst2 = input('Wpisz jeszcze coś: ')
if tekst1 == tekst2:
    print('Wpisano dwa razy to samo:')
    print('tekst', tekst1)
```

```
Wpisz cokolwiek: łoś
Wpisz jeszcze coś: łoś
Wpisano dwa razy to samo:
tekst łoś
```

Proszę sobie dobrze zapamiętać :-)

= przypisuje

== porównuje

Operatory porównania

Dzięki operatorowi != program też porównuje argumenty tego operatora. Wynik operatora != to:
+ **True**, gdy oba jego argumenty nie są równe, czyli są różne
+ **False**, gdy oba jego argumenty są równe

Operator != czytamy „nie równa się”

Operatory porównania

```
tekst1 = input('Wpisz cokolwiek: ')
tekst2 = input('Wpisz jeszcze coś: ')
if tekst1 != tekst2:
    print('Wpisano dwa razy co innego:')
    print('tekst', tekst1, 'i tekst', tekst2)
```

```
Wpisz cokolwiek: łoś
Wpisz jeszcze coś: dzik
Wpisano dwa razy co innego:
tekst łoś i tekst dzik
```

Operatory in i not in

Operator in

Dzięki operatorowi in program sprawdza, czy jeden łańcuch zawiera się w drugim łańcuchu. Wynik operatora in to:
+ **True**, gdy jego lewy argument zawiera się w jego prawym argumencie
+ **False**, gdy jego lewy argument nie zawiera się w jego prawym argumencie

Operator in czytamy „zawiera się w"

Operator in

```
tekst = input('Wpisz cokolwiek: ')
if 'łoś' in tekst:
    print('Wpisano tekst', tekst)
    print('W tekście', tekst, 'jest wyraz "łoś"')
```

```
Wpisz cokolwiek: zażyłość
Wpisano tekst zażyłość
W tekście zażyłość jest wyraz "łoś"
```

Operator not in

Dzięki operatorowi not in program też sprawdza, czy jeden łańcuch zawiera się w drugim łańcuchu. Wynik operatora not in to:

+ True, gdy jego lewy argument zawiera się w jego prawym argumencie
+ False, gdy jego lewy argument nie zawiera się jego prawym argumencie

Operator not in czytamy „nie zawiera się w"

Operator not in

```
tekst = input('Wpisz cokolwiek: ')
if ' ' not in tekst:
    print('Wpisano tekst', tekst)
    print('W tekście', tekst, 'nie ma spacji')
```

```
Wpisz cokolwiek: dzik
Wpisano tekst dzik
W tekście dzik nie ma spacji
```

Operatory logiczne

Operator logiczny or

Dzięki operatorom logicznym jedna instrukcja warunkowa sprawdza wiele warunków naraz

Operator or czytamy „lub". Wynik operatora or to:
+ True, gdy dowolny z jego dwóch argumentów ma wartość True
+ False, gdy oba jego argumenty mają wartość False

Operator logiczny or

```
litera = input('Wpisz symbol zasady: ')
if litera == 'A' or litera == 'G':
    print('To zasada purynowa')
if litera == 'C' or litera == 'T' or litera == 'U':
    print('To zasada pirymidynowa')
Wpisz symbol zasady: U
To zasada pirymidynowa
```

Operator logiczny and

Operator and czytamy „i”. Wynik operatora and to:
+ True, gdy oba jego argumenty mają wartość True
+ False, gdy dowolny z jego argumentów ma wartość False

```
hasło = input('Podaj hasło: ')
if len(hasło) >= 12 and '!' in hasło:
    print('Masz silne hasło :-)')
```

```
Podaj hasło: nie,xo338j!d32,zd
Masz silne hasło :-)
```

Teraz nauczymy się czegoś nowego:
operatora jednoargumentowego :-)

Operator logiczny not

Operator not czytamy „nie”
Operator not jest jednoargumentowy, ma tylko prawy argument
Wynik operatora not to:
+ False, gdy jego prawy argument ma wartość True
+ True, gdy jego prawy argument ma wartość False

Operator logiczny not

```
hasło = input('Podaj hasło: ')
if not hasło.startswith('*') and not hasło.endswith('*'):
    print('Hasło nie zaczyna się od znaku * i nie kończy się znakiem *')
```

```
Podaj hasło: nie,xo338j!d32,zd
Hasło nie zaczyna się od znaku * i nie kończy się znakiem *
```

Operator logiczny **not**

```
tekst = 'Python to łatwy język programowania'
if 'język' in tekst and not tekst.startswith('język'):
    print('Tekst zawiera wyraz "język", ale się od niego nie zaczyna')
```

```
Tekst zawiera wyraz "język", ale się od niego nie zaczyna
```

Z operatorów logicznych **or**, **and** i **not** można zbudować dowolne wyrażenie logiczne

Instrukcja warunkowa **if**, operatory porównania == i !=, operatory **in** i **not in**, operatory logiczne **or**, **and** i **not**
Powtórzenie

Instrukcja warunkowa, operatory ==, !=, in, not in, or, and i not

```
hasło = input('Podaj hasło: ')
if hasło == '':
    print('Puste hasło')
if hasło != 'xyzzy':
    print('Niepoprawne hasło')
if ' ' in hasło:
    print('Hasło zawiera spację')
if ' ' not in hasło:
    print('Hasło nie zawiera spacji')
```

```
if ' ' in hasło or '\t' in hasło:
    print('Hasło zawiera biały znak')
if '*' in hasło and '#' in hasło:
    print('Bardzo silne hasło :-)')
if ('xyzzy' in hasło and
     not hasło.startswith('xyzzy')):
    print('W haśle ukryto "xyzzy"')
```

Postęp wykładu

Przedstawiłem wam:
+ Funkcję **print**
+ Funkcje
+ Łańcuchy znaków, czyli teksty
+ Instrukcję przypisania
+ Słowa kluczowe
+ Funkcję **input**
+ Indeksowanie łańcuchów
+ Wycinki łańcuchów
+ Dodawanie łańcuchów
+ Metody łańcuchów
+ Instrukcję warunkową **if**
+ Operatory porównania
+ Operatory **in** i **not in**
+ I operatory logiczne

Teraz omówię:
+ Pliki tekstowe

Proszę mi zadać pytanie :-)

Wróćmy do zadania na rozgrzewkę

Gdybym dostał zadanie „proszę narysować plik”, narysowałbym plik kartek

A wy?



Jestem programistą od 30 lat

"Open file" i "close file" tłumaczy się „otwórz plik" i „zamknij plik". Nie umiałem sobie tego wyobrazić…



...aż do stycznia 2025 r. Zwiedzałem wtedy Muzeum Czartoryskich

Zobaczyłem eksponat podpisany po angielsku „File of Stanislaus August Poniatowski" i tak się ucieszyłem, że aż podskoczyłem z radości



Polski podpis brzmiał: „Teczka Stanisława Augusta Poniatowskiego" :-)

Już wiecie, jak wygląda plik. „Plik" to nietrafne tłumaczenie wyrazu „file". Niestety jest za późno na to, by je zmienić :-)



Pliki tekstowe

Pliki tekstowe to takie pliki, które są podzielone na linie

Linie są oddzielone znakami końca linii. W Pythonie oznacza się to przez \n

Nowak\n
Wójcik\n
Kowalska\n
Kowalski\n

(Oprócz plików tekstowych są pliki binarne, o których nic więcej nie powiem)

Otwieranie pliku — funkcja **open**

Zanim program zacznie odczytywać plik, musi otworzyć ten plik funkcją **open**

+ pierwszy argument funkcji **open** to nazwa pliku
+ drugi argument funkcji **open** to tryb otwarcia pliku:
'r' jak „read" albo 'w' jak „write"

Otwieranie pliku — funkcja **open**

Funkcja **open** zwraca plik otwarty do odczytu albo do zapisu. Na zajęciach z tego przedmiotu będziemy otwierać pliki tylko do odczytu

(Programista może pominąć drugi argument funkcji **open**. Wtedy Python domyśli się, że programista miał na myśli 'r'. Drugi argument funkcji **open** ma wartość domyślną 'r')

Odczytywanie linii z pliku tekstowego — pętla for

Tak program wypisuje na ekranie linię po linii z pliku tekstowego otwartego do odczytu:

```
for linia in otwarty_plik:
    print(linia)
```

Pamiętajmy, że po każdym dwukropku trzeba zrobić wcięcie

Odczytywanie linii z pliku tekstowego — pętla for

Oto wynik działania tego programu:

```
for linia in otwarty_plik:
    print(linia)
```

```
Aaron m
Abba m
Abdon m
Abel m
    i tak dalej…
```

Zamykanie pliku tekstowego

Kiedyś trzeba było pisać tak:

```
plik = open('nazwa-konkretnego-pliku.txt', 'r')
for linia in plik:
    print(linia)
plik.close()
```

Wtedy było łatwo zapomnieć o tym, żeby zamknąć plik

Zamykanie pliku tekstowego

Już nie trzeba pamiętać o tym, żeby zamknąć plik :-)

Zalecam pisać tak:

```
with open('nazwa-konkretnego-pliku.txt', 'r') as plik:
    for linia in plik:
        print(linia)
```

Przetwarzanie pliku tekstowego

Pamiętajmy, że możemy zagnieżdżać wcięcia. Jeśli linia programu zaczyna się wcięciem i kończy się dwukropkiem, to następna linia zaczyna się większym wcięciem

Oto przykład programu, który przetwarza plik tekstowy:

```
with open('imiona.txt', 'r') as plik:
    for linia in plik:
        if linia.startswith('Dag'):
            print(linia)
```

```
Dagmara ż
Dagna ż
Dagobert m
```

Przetwarzanie pliku tekstowego

Oto inny przykład programu, który przetwarza plik tekstowy.
Zauważ, że kod pętli **for** może zawierać więcej niż jedną instrukcję

```
with open('imiona.txt', 'r') as plik:
    for linia in plik:
        if linia.endswith('ż'):
            print('kobieta')
        if linia.endswith('m'):
            print('mężczyzna')
```

Przedstawiłem wam:

+ Funkcję **print**
+ Funkcje
+ Łańcuchy znaków, czyli teksty
+ Słowa kluczowe
+ Instrukcję przypisania
+ Funkcję **input**
+ Indeksowanie łańcuchów
+ Wycinki łańcuchów
+ Dodawanie łańcuchów
+ Metody łańcuchów
+ Instrukcję warunkową **if**
+ Operatory porównania
+ Operatory **in** i **not in**
+ Operatory logiczne
+ I pliki tekstowe

To wszystko na dziś
Dziękuję wam za uwagę :-)

Proszę mi zadać pytanie :-)

Czy ktoś z was czegokolwiek
nie zrozumiał z tego wykładu?

Do zobaczenia
na następnym wykładzie,
na którym przykłady będą po czesku :-)

Źródła zdjęć

+ Microsoft Designer: jabłko z etykietką, gruszka z etykietką, gruszka z dwiema etykietkami, plik kartek
+ Pałac Czartoryskich, Wikimedia Commons, CC BY-SA 4.0
+ Muzeum Narodowe w Krakowie, teczka Stanisława Augusta Poniatowskiego, domena publiczna